

Spring security

What is it?

- powerful and highly customizable framework for securing Java applications, especially those built with Spring.
- **What does it do?**
 - Provides comprehensive features for both **authentication** (who the user is) and **authorization** (what the user is allowed to do).
- **How does it work?**
 - The core functionality is based on a chain of servlet filters that intercepts every incoming HTTP request.
- **Why use it?**
 - It is the industry standard for securing Spring applications and offers built-in protection against many common exploits like CSRF, session fixation, and clickjacking.

- Provides sensible defaults and auto-configuration.

Key purpose:

- Authentication: Verifies "who" a user is.
- Authorization: Determines "what" an authenticated user is allowed to do.
- Why use it with Spring Boot?
 - Adds security with minimal code.
 - Provides sensible defaults and auto-configuration.

- **Authentication:** Verifying the identity of a user (e.g., username/password login).
- **Authorization:** Determining what an authenticated user is allowed to do (e.g., access specific resources, perform certain actions).
- **Principal:** Represents the currently authenticated user.
- **SecurityContext:** Stores the security information of the current principal.
-

Maven dependency

```
<dependency>
```

```
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-security</artifactId>
```

```
</dependency>
```

- Default Security:
 - Spring Boot automatically configures basic in-memory authentication with a generated password upon adding the dependency.
- Customizing Default User:
 - Configure `spring.security.user.name` and `spring.security.user.password` in `application.properties` for custom in-memory credentials.

Basic Configuration

- `WebSecurityConfigurerAdapter` (Older versions) / `SecurityFilterChain` (Newer versions)
- `@EnableWebSecurity`: Enables Spring Security in your application.
- Configuring Authentication: Define `UserDetailsService` or `AuthenticationProvider` to specify how users are loaded and authenticated (e.g., in-memory, JDBC, LDAP).
- Configuring Authorization: Define access rules for different URL patterns or methods using `http.authorizeRequests()`.

User, UserDetails, UserDetailsManager

- In Spring Security, the components User, UserDetails, UserDetailsService, and UserDetailsManager play crucial roles in handling user authentication and authorization.
- User (org.springframework.security.core.userdetails.User):
 - This is a concrete implementation of the UserDetails interface provided by Spring Security. It's a convenient class to use when you don't need to add custom user properties beyond the basic ones like username, password, and authorities.
- UserDetails (interface org.springframework.security.core.userdetails.UserDetails):
 - This interface represents the core user information that Spring Security uses for authentication and authorization.
 - It defines methods to retrieve the user's username, password, authorities (roles), account expiration status, account locking status, credentials expiration status, and account enabled status.
- UserDetailsService (interface org.springframework.security.core.userdetails.UserDetailsService):
 - This is a crucial interface for retrieving user-specific data during authentication. It has a single method, loadUserByUsername(String username), which is responsible for fetching a UserDetails object based on the provided username.
 - You implement this interface to define how Spring Security retrieves user information from your chosen data source (e.g., database, LDAP, in-memory store).
- UserDetailsManager (interface org.springframework.security.provisioning.UserDetailsManager):
 - This interface extends UserDetailsService and provides additional methods for managing users, including creating, updating, and deleting users, as well as changing passwords. It's useful when you need to perform CRUD operations on user accounts within your application, often backed by a persistent user repository like a database.

- Relationship:

- The UserDetailsService is responsible for loading a UserDetails object based on a username.
- The User class is a default implementation of the UserDetails interface.
- The UserDetailsManager extends UserDetailsService and adds capabilities for managing user accounts.
- UserDetailsService focuses on retrieving user details for authentication.
- UserDetailsManager provides a more comprehensive set of operations for managing user accounts, including their creation and modification.

InMemoryUserDetailsManager

- Spring Security uses InMemoryUserDetailsManager to manage these in-memory user details.
- This class implements UserDetailsService and UserDetailsManager, allowing it to retrieve and manage user information stored in memory.
- Password Encoding:
 - While withDefaultPasswordEncoder() can be used for simplicity in development, it is not suitable for production.
 - In production environments, stronger password encoders like BCryptPasswordEncoder should be used to securely hash passwords.

example

@Configuration

@EnableWebSecurity

public class SecurityConfig {

@Bean

public UserDetailsService userDetailsService() {

 UserDetails user = User.withDefaultPasswordEncoder()

 .username("user")

 .password("password")

 .roles("USER")

 .build();

 UserDetails admin = User.withDefaultPasswordEncoder()

 .username("admin")

 .password("adminpassword")

 .roles("ADMIN")

 .build();

 return new InMemoryUserDetailsManager(user, admin);

}

@Bean

public SecurityFilterChain filterChain(HttpSecurity http) throws
Exception {

 http

 .authorizeHttpRequests(authorize -> authorize

 .requestMatchers("/admin").hasRole("ADMIN")

 .requestMatchers("/user").hasAnyRole("USER", "ADMIN")

 .anyRequest().authenticated()

)

 .formLogin(Customizer.withDefaults());

 return http.build();

}

}

The Security Filter Chain

- The Foundation: Spring Security's web support is built on standard servlet filters, which are executed in a specific order for each request.
- Intercepting Requests:
 - A special filter, `FilterChainProxy`, manages the entire security filter chain.
 - It contains multiple `SecurityFilterChain` instances, each configured to a specific URL pattern.
- Key Filters in the Chain:
 - `SecurityContextPersistenceFilter`: Sets up the `SecurityContextHolder` at the beginning of a request.
 - Authentication filters (e.g., `UsernamePasswordAuthenticationFilter`): Intercept login requests and delegate to the `AuthenticationManager`.
 - `FilterSecurityInterceptor`: Performs authorization based on the secured URL.
 - Order Matters: The order of filters is crucial. For example, authentication must happen before authorization.

Using filter

@Configuration

@EnableWebSecurity

public class SecurityConfig{

@Bean

public UserDetailsService userDetailsService(){

 UserDetails user = User.withDefaultPasswordEncoder()

 .username("user")

 .password("password")

 .roles("USER")

 .build();

 return new InMemoryUserDetailsManager(user);

}

@Bean

public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {

 http

 .authorizeHttpRequests((requests) -> requests

 .requestMatchers("/", "/home").permitAll()

 .anyRequest().authenticated()

)

 .formLogin((form) -> form

 .loginPage("/login")

 .permitAll()

)

 .logout((logout) -> logout.permitAll());

 return http.build();

}

}

Customizer

- Customizer is a functional interface used to apply custom configurations to various components within the security setup.
- It provides a concise and flexible way to modify default behaviors or add specific configurations without needing to extend or override entire classes.
 - Functional Interface: Customizer is a functional interface with a single abstract method, `customize(T t)`. This allows it to be used with lambda expressions, making configurations more readable and compact.
 - Applying Customizations: Use Customizer to modify the behavior of specific Spring Security components. For example, you can use it with `httpBasic()`, `formLogin()`, `csrf()`, or `logout()` to customize their respective configurations.

@Bean

```
public SecurityFilterChain securityFilterChain(HttpSecurity http)
throws Exception {
    http
        .csrf(csrf -> csrf.disable()) // Using a Customizer with a lambda
        // ... other configurations
        .build();
    return http.build();
}
```

`csrf -> csrf.disable()` is a lambda expression implementing the `Customizer<CsrfConfigurer<HttpSecurity>>` interface.

`withDefaults()`: The `Customizer.withDefaults()` static method provides a `Customizer` that applies the default configuration for a given component. This is useful when you want to explicitly indicate that you are accepting the default behavior without any custom modifications.

@Bean

```
public SecurityFilterChain securityFilterChain(HttpSecurity  
httpSecurity) throws Exception {  
    httpSecurity.httpBasic(Customizer.withDefaults());  
    httpSecurity.authorizeHttpRequests(request-  
>request.requestMatchers("/","/home").permitAll().anyRequest().au  
thenticated());  
    return httpSecurity.build();  
}
```


- Enabling Spring Security with `@Configuration` and `@EnableWebSecurity`.
- Defining authorization rules using `authorizeHttpRequests()` within the `SecurityFilterChain` bean.
- `requestMatchers()` is used for path-based access control, allowing public access with `permitAll()` or restricting access by role with `hasRole()`.
- `anyRequest().authenticated()` secures all other endpoints.
- Enabling HTTP Basic authentication with `httpBasic()`, which prompts for credentials.
- Disabling CSRF protection, often done for stateless REST APIs.
- Setting up in-memory users with `InMemoryUserDetailsManager`.
- Using `PasswordEncoderFactories.createDelegatingPasswordEncoder()` for secure password handling.

The Authentication Flow

Step 1: Request Arrives: An HTTP request hits the web application and is intercepted by the security filter chain.

Step 2: Credential Extraction: An authentication filter (e.g., `UsernamePasswordAuthenticationFilter`) extracts the user's credentials, such as a username and password, from the request.

Step 3: Authentication Token: The filter creates an unauthenticated `Authentication` object, typically a `UsernamePasswordAuthenticationToken`, with the credentials.

Step 4: Authentication Attempt: The `AuthenticationManager` attempts to authenticate the token.

Step 5: Provider Authentication: The `AuthenticationManager` delegates the task to a list of `AuthenticationProviders`.

Step 6: User Details Loading: A `DaoAuthenticationProvider` uses a `UserDetailsService` to retrieve user information from a database and a `PasswordEncoder` to verify the password.

Step 7: Context Update: If authentication is successful, the provider returns a fully populated `Authentication` object, which is stored in the `SecurityContext` managed by the `SecurityContextHolder`.

Authorization and Access Control

- Post-Authentication: After a user is authenticated, authorization determines what they are allowed to do.
- Role-Based Access Control (RBAC): The most common authorization model, where permissions are granted based on a user's assigned roles (e.g., ROLE_ADMIN, ROLE_USER).
- URL-Based Security: Access to specific URLs or paths is restricted based on an authenticated user's roles.
- Method-Level Security: Annotations like `@PreAuthorize` can be used to protect specific methods within a service layer.
- Decision Making: The `FilterSecurityInterceptor` uses an `AuthorizationManager` to

- **SecurityContextHolder:** A central object that stores the details of the currently authenticated user for the duration of a request.
- **Authentication:** An object that represents the user's principal (the user), credentials, and granted authorities.
- **UserDetailsService:** An interface used to retrieve user-specific data from a data source.
- **UserDetails:** An object representing the user's details, including username, password, roles, and account status.
- **PasswordEncoder:** A component for securely hashing and comparing passwords.
- **GrantedAuthority:** Represents a permission or role granted to a user.

AuthenticationProvider

- AuthenticationProvider is a component that verifies a user's identity during the authentication process.
- It validates a user's credentials against a database or external identity source to confirm that they are who they claim to be.
- The most common example is authenticating a user with a username and password, but providers can be designed for other mechanisms like JSON Web Tokens (JWT) or LDAP

Common types of AuthenticationProvider in Spring

- DaoAuthenticationProvider: Authenticates a username and password against a database by using a UserDetailsService to load user information.
- LdapAuthenticationProvider: Authenticates a user against an LDAP (Lightweight Directory Access Protocol) server.
- OAuth2LoginAuthenticationProvider: Handles authentication for OAuth 2.0 logins, such as "Login with Google" or "Login with Facebook".
- JwtAuthenticationProvider: Authenticates users based on JSON Web Tokens (JWT) for use in OAuth 2.0 Resource Servers.
- RememberMeAuthenticationProvider: Uses previously saved "remember me" tokens to automatically re-authenticate a user.

BCryptPasswordEncoder

- is a password encoder implementation provided by the Spring Security framework that uses the bcrypt hashing algorithm.
- Its purpose is to securely store user passwords by converting them into a hashed, irreversible string

Role-Based Access Control (RBAC)

- It is a security model implemented using Spring Security to manage authorization based on user roles.
- Instead of granting permissions directly to individual users, RBAC assigns permissions to roles, and then users are assigned to those roles.
- Key Concepts in RBAC with Spring Boot:
 - Users: Individuals who interact with the application and are authenticated.
 - Roles: Collections of permissions assigned to a group of users based on their responsibilities or tasks (e.g., ADMIN, USER, MANAGER).
 - Permissions: Specific rights or actions that a user can perform within the application (e.g., read, write, delete, update).

Map Users to Roles:

- Associate users with their respective roles. This can be done during user creation or through an administrative interface.
- When a user authenticates, their assigned roles are loaded and used for authorization decisions.
- Implement Authorization Logic:
 - Method-level security: Use annotations like `@PreAuthorize("hasRole('ROLE_ADMIN')")` or `@PreAuthorize("hasAuthority('permission:read')")` on controller methods or service methods to restrict access based on the authenticated user's roles or permissions.
 - Ensure `@EnableMethodSecurity` (or `@EnableGlobalMethodSecurity` for older versions) is enabled in your security configuration.

Define user and role entities

Role.java

java

```
public class Role implements GrantedAuthority {
```

```
    private String name;
```

```
    public Role(String name) {
```

```
        this.name = name;
```

```
    }
```

```
    @Override
```

```
    public String getAuthority() {
```

```
        return name;
```

```
    }
```

```
    // Getters and setters...
```

```
}
```

```
public class User implements UserDetails {

    private String username;
    private String password;
    private List<Role> roles;

    // A real-world application would use a database and ORM
    public User(String username, String password, List<Role>
roles) {
        this.username = username;
        this.password = password;
        this.roles = roles;
    }

    @Override
    public Collection<? extends GrantedAuthority>
getAuthorities() {
        return this.roles;
    }
}
```

```
// Additional UserDetails interface methods...

@Override
public String getUsername() {
    return username;
}

@Override
public String getPassword() {
    return password;
}

// Implement other UserDetails methods
(isAccountNonExpired, etc.).
// For simplicity, we can return true for now.
}
```

CustomUserDetailsService

@Service

```
public class CustomUserDetailsService  
implements UserDetailsService {
```

```
    private final Map<String, User> users = new  
    HashMap<>();
```

@PostConstruct

```
    public void init() {  
        users.put("admin", new User("admin",  
            "{noop}password", List.of(new  
            Role("ROLE_ADMIN"), new Role("ROLE_USER"))));  
        users.put("user", new User("user",  
            "{noop}password", List.of(new  
            Role("ROLE_USER"))));  
    }
```

@Override

```
    public UserDetails loadUserByUsername(String  
    username) throws UsernameNotFoundException {  
        User user = users.get(username);  
        if (user == null) {  
            throw new  
            UsernameNotFoundException("User not found");  
        }  
        return user;  
    }  
}
```

Configure SecurityFilterChain

@Configuration

@EnableWebSecurity

```
public class SecurityConfig {

    private final CustomUserDetailsService userDetailsService;

    public SecurityConfig(CustomUserDetailsService
userDetailsService) {

        this.userDetailsService = userDetailsService;
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
throws Exception {

        http

            .authorizeHttpRequests(authorize -> authorize

                .requestMatchers("/admin").hasRole("ADMIN")

                .requestMatchers("/user").hasAnyRole("ADMIN", "USER")

                .requestMatchers("/", "/public").permitAll()

                .anyRequest().authenticated()

            )
```

```
.userDetailsService(userDetailsService)
```

```
        .formLogin(Customizer.withDefaults()); // Use
default form login
```

```
        return http.build();
```

```
    }
```

// To use in-memory users for basic testing, you can override the default.

// In a real app, the UserDetailsService bean and UserDetails implementation would suffice.

// @Bean

```
// public UserDetailsService users() {
```

```
    // UserDetails admin =
User.builder().username("admin").password("{noop}pas
sword").roles("ADMIN", "USER").build();
```

```
    // UserDetails user =
User.builder().username("user").password("{noop}passw
ord").roles("USER").build();
```

```
    // return new InMemoryUserDetailsManager(admin,
user);
```

```
    // }
```

```
}
```

- `hasRole()` and `hasAuthority()`:
 - The `hasRole()` method is a shortcut that automatically prefixes the role name with "ROLE_", so `hasRole("ADMIN")` looks for the authority "ROLE_ADMIN".
- Request matchers:
 - Rules should be ordered from most specific to least specific. The `anyRequest().authenticated()` at the end acts as a catch-all.
- `{noop}` prefix:
 - This tells Spring to use a plain-text password for demonstration. In a production application, you should always use a password encoder like `BCryptPasswordEncoder`.

Controller

```
@RestController
public class TestController {

    @GetMapping("/public")
    public String publicEndpoint() {
        return "This is a public endpoint.";
    }

    @GetMapping("/admin")
    public String adminEndpoint() {
        return "This is an ADMIN-only endpoint.";
    }

    @GetMapping("/user")
    public String userEndpoint() {
        return "This is a USER endpoint, accessible by ADMINS
and USERS.";
    }
}
```

Method-level security with @PreAuthorize

- Enable method security: Add the @EnableMethodSecurity annotation to your SecurityConfig class.SecurityConfig.java

@Configuration

@EnableWebSecurity

@EnableMethodSecurity // Add this line

```
public class SecurityConfig {
```

```
    // ... (rest of the class is the same)
```

```
}
```


- Use annotations on methods: Apply annotations like @PreAuthorize to your controller or service methods.

@RestController

```
public class TestController {
```

```
    // ... (other endpoints)
```

```
    @GetMapping("/admin/data")
```

```
    @PreAuthorize("hasRole('ADMIN')")
```

```
    public String adminData() {
```

```
        return "This data is only for ADMINS.";
```

```
    }
```

```
    @GetMapping("/user/data")
```

```
    @PreAuthorize("hasRole('USER')")
```

```
    public String userData() {
```

```
        return "This data is for USERS.";
```

```
    }
```

```
}
```